

Module 5

Backtracking : n queens problem
Subset-Sum problem

Branch & Bound
Knapsack
Travelling Salesman

NP completeness

Polynomial time

Polynomial time Verification

NP Completeness & reducibility

NP Complete problems:

clique problem

Vertex Cover problem.

Backtracking :

The idea is to construct some one component at a time and evaluate such partially constructed candidates as follows:

- * If a partially constructed some can be developed without violating problem constraint, it is done by taking the first remaining legitimate options for next component.
- * If there is no legitimate options algorithm backtracks to replace the first component of the partially constructed some with the next option.

State-space tree is ^{used} to implement backtracking by constructing a tree of choices being made.

→ It ~~not~~ represents an initial state before the search for solution begins.

→ Nodes of first level in the tree represent the choice made for the first component of solution.

→ Nodes of second level represent the choices for second component and so on.

→ A Node in a state-space tree is said to be promising if it corresponds to a partially constructed solution that may still lead to complete solution, otherwise it is nonpromising.

N-Queens Problem

Problem is to place N -queens on a n -by- n chess board so that no two queens attack each other by being in the same row, or same column or on the same diagonal.

Eg: Design:- 4×4 Queen

	1	2	3	4	
1	.				← queen 1
2	.				← queen 2
3	.				← queen 3
4	.				← queen 4

Board for four-Queen problem

Possible Soln

①

	1	2	3	4
1		Q ₁		
2				Q ₂
3	Q ₃			
4			Q ₄	

②

	1	2	3	4
1			Q ₁	
2	Q ₂			
3				Q ₃
4		Q ₄		

we will implement n-queen using
Backtracking by considering state-space tree

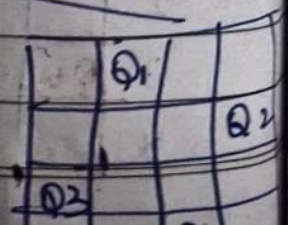
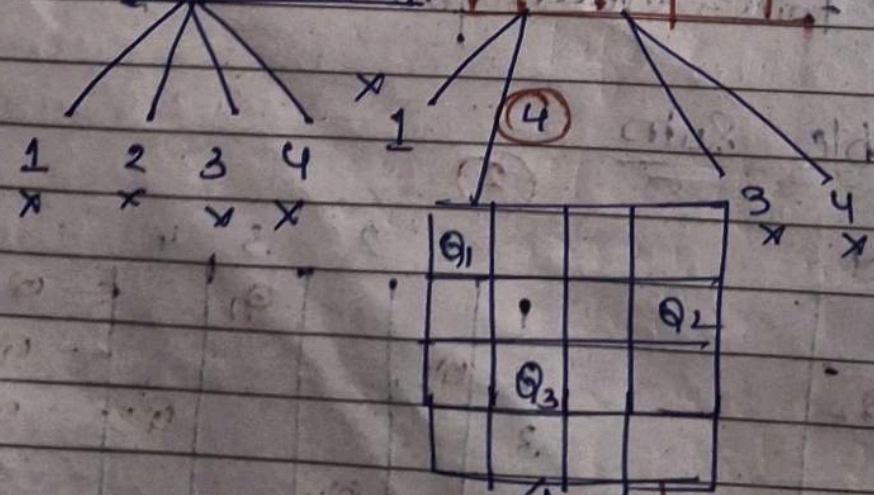
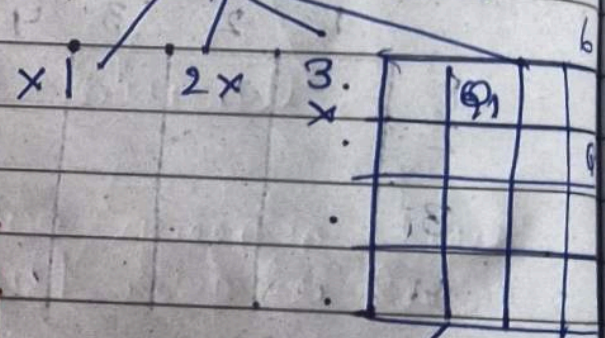
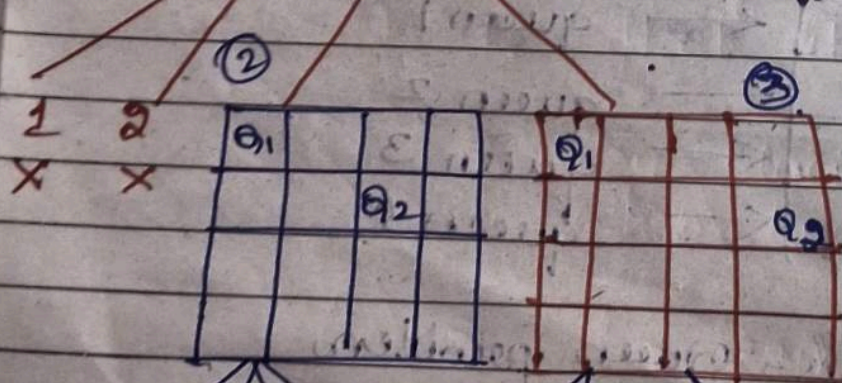
Implement 4x4 N-queen by considering state
space tree

So



S1

5



Final soln by this state-space tree

	1	2	3	4
1		Q_1		
2				Q_2
3	Q_3			
4			Q_4	

$Q_1 \rightarrow (1, 2)$

$Q_2 \rightarrow (2, 4)$

$Q_3 \rightarrow (3, 1)$

$Q_4 \rightarrow (4, 3)$

Time Complexity

Finally, it should be pointed out that a single solution to n queens problem for any $n \geq 4$ can be found in linear time.

Solution for 8 queen will be

		Q_1					
				Q_2			
	Q_3						
					Q_4		
Q_5							
			Q_6				
						Q_7	
				Q_8			

Solution

Algorithm for N-queens

Step 1 :- Check whether two queens attack each other by checking if they are in same row / column / diagonal.

~~They~~ →

if $\text{board}[i] == \text{board}[\text{current_column}]$
same row.

if $\text{abs}(\text{board}[i] - \text{new_row}) == \text{abs}(i - \text{current_column})$
same diagonal.

Step 2 :- Backtracking

1. Start at column 0.

2. Try placing queen in Row 0, then Row 1, up to Row N-1.

3. for each row check if it is safe.

4. If safe: lock it in the array and move to next column.

5. If Not Safe / Dead: change the row value and try next number.

Subsum - Sum problem

To find a subset of given set

$A = \{a_1, \dots, a_n\}$ of n positive integers

whose sum is equal to given positive

integer d .

Eg for set $A = \{1, 2, 5, 6, 8\}$
and $d = 9$.

There are 2 possible sols

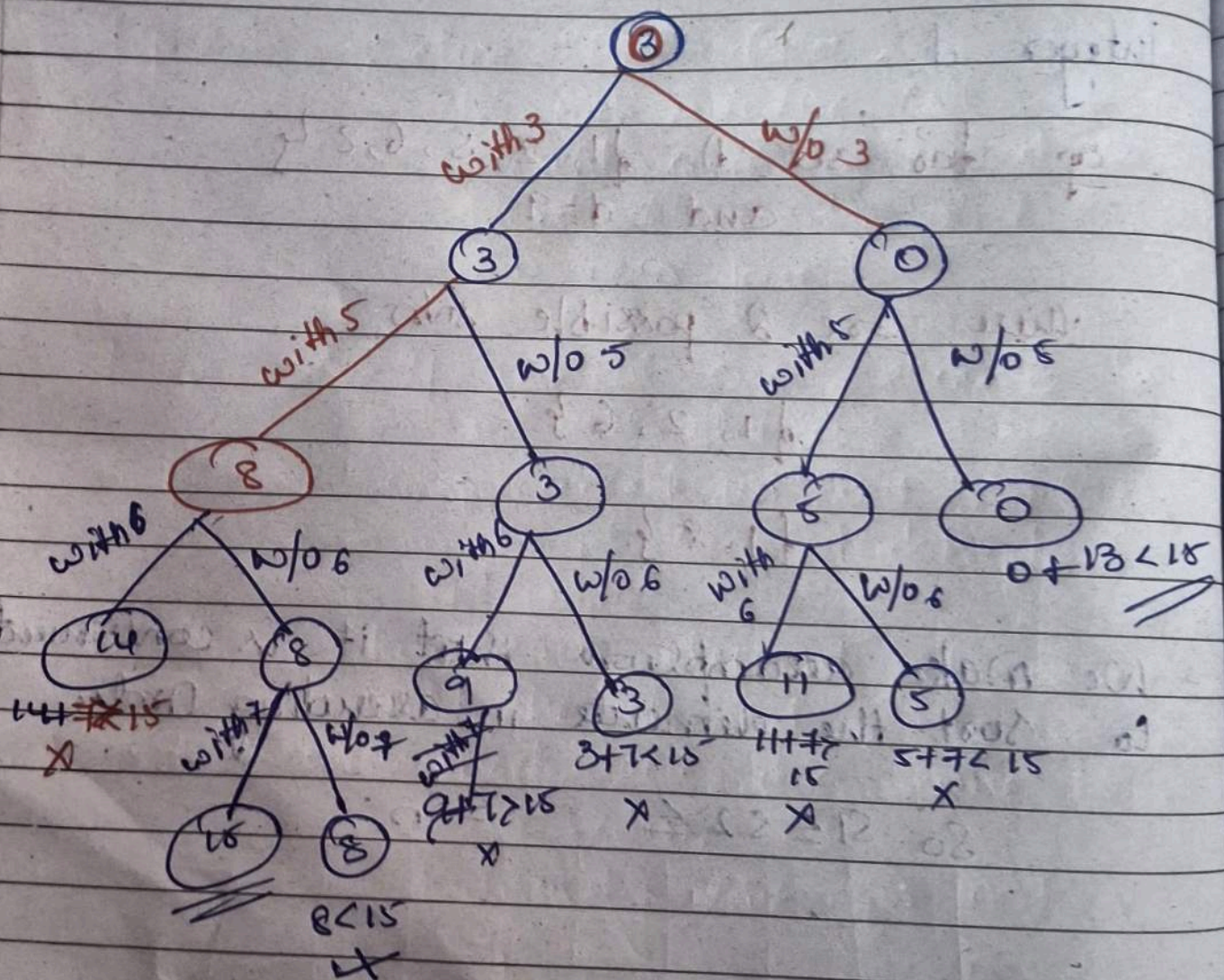
$\{1, 2, 6\}$

$\{1, 8\}$

We make assumption that it is convenient
to sort the elements in ascending order

$$\text{So } s_1 \leq s_2 \leq \dots \leq s_n$$

Complete the state space tree for
backtracking algorithm applied to
 $A = \{3, 5, 6, 7\}$ and $d = 15$ of Subset-sum
problem



So the subset for $d = 15$
is $\{3, 5, 7\}$
d

Note:

If s is equal to d we have soln to problem.

we can either report this result and stop or if all soln needed to found

continue by backtracking to node's parent

If x_i is not equal to d , we terminate the node as nonpromising if either of

the following two inequalities holds:

$$s + a_{ij} > d \quad (\text{the sum } s \text{ is too large})$$

$$s + \sum_{j=i+1}^n a_j < d \quad (\text{the sum } s \text{ is too small})$$

Algorithm Steps

Step 1: Initialize start :- start with subset of current sum 0 i.e. first index of array.

Step 2: Define Base Cases
In each recursive call, If current sum is equal to target-sum, a valid subset has been found.

If current sum exceeds target or all elements have been processed without reaching target sum backtrack.
1st choice

Step 3: For each element of current index Add the element to the current subset and its value to current sum.

Recursively call for next index

2nd choice:- Remove the element and recursively call the function for next index without adding value.

i.e

Algorithm FindSet(set, n, index, current sum, target sum) {

if (current sum == target sum)

{

return 1;

}

if (current sum > target sum || index >= n)

{

return 0;

}

if (findSet(set, n, index+1, current sum + set[index], target sum)

{

printf("%d", set[index]);

return 1

}

~~if (findSet(set, n,~~

return findSet(set, n, index+1, current sum, target sum)

}

Algorithm takes $O(2^n)$ time

Branch and Bound